

10) Data Model and Storage

Last updated by | Mennell, Jacob | 19 May 2026 at 15:39 BST

Data Model And Storage

Purpose

This page describes the main persisted entities used by the solution and explains how operational data, chunked evidence, and feedback are stored.

Scope

The solution uses a data model centered around claims, documents, chunked evidence, workflow outputs, access control records, and feedback. PostgreSQL is the primary store, and pgvector extends it with vector similarity support for retrieval.

Storage Approach

The solution uses PostgreSQL as the main application database and pgvector for embedding storage and similarity search. This keeps operational records and retrieval-ready vector data in one persistence layer.

This is a pragmatic design because:

- business entities and retrieval entities remain closely linked
- filtering and retrieval scope can be enforced in the same database layer
- orchestration and API services can rely on a shared source of truth

Core Entity Groups

Claims

Claims are the primary business entities in the system. API routes show claim retrieval, ownership updates, and summary views. Claim records also appear to carry structured data and metadata that can be extended with additional history or workflow outputs.

Documents

Documents are attached to claims and carry business-facing attributes such as document type, document name, document date, source URI, and ingestion state. They act as the parent entities for chunk persistence.

Document chunks

Document chunks are the primary retrieval units. Each chunk record stores:

- `chunk_id`
- `document_id`
- `claim_id`
- `document_type`
- `chunk_index`
- `text_chunk`
- `embedding`

- `meta_data`
- `optional chunk_context`

This makes the chunk table the bridge between ingestion, retrieval, and orchestration.

Chunk metadata

Chunk metadata is stored as JSON and, based on the schema, can contain:

- chunk hash
- semantic source
- header hierarchy and flattened header path
- section identifiers
- page and layout metadata
- geometry data
- schema versioning information

This provides a flexible extension point without requiring a schema change for every metadata addition.

Feedback records

The feedback routes and model definitions show a dedicated module feedback entity for capturing check-specific reviewer feedback. This includes conclusive answers, explanatory text, leakage-related fields, expert appointment fields, timestamps, and user identity.

This is a useful separation because feedback is operationally important but should not be conflated with the base claim or chunk storage models.

Access and model assignment records

The model access endpoints indicate dedicated records for mapping users to models, including active status and user attributes. This supports role- and model-driven access patterns in the application.

Why Pgvector Matters In This Model

The retrieval layer depends on dense vector similarity over chunk embeddings. By storing the embedding alongside the chunk record, the system can:

- filter by claim and document scope inside SQL
- retrieve semantically similar chunks without exporting data to an external vector system
- keep lexical and semantic retrieval anchored to the same underlying entity set

This supports a tighter, more controlled retrieval architecture than splitting operational and vector storage into unrelated systems.

Data Model Relationships



flowchart TD

```
Claim[Claim] --> Document[Document]
Document --> Chunk[Document Chunk]
Chunk --> Meta[Chunk Metadata JSON]
Chunk --> Vec[Vector Embedding]
Claim --> Feedback[Module Feedback]
Model[Model] --> ModelUser[Model User Access]
Claim --> Model
```

Design Decisions

- Claims and documents remain the parent business entities.
- Chunks are stored as first-class retrieval entities, not computed ad hoc.
- Metadata is partly relational and partly JSON-based to balance structure with flexibility.
- Feedback and access control are stored separately from claim core data.
- PostgreSQL plus pgvector keeps operational and retrieval data co-located.

Operational Notes

- Ingestion completeness directly affects the availability of chunk records and embeddings.
- Retrieval quality depends on chunk text, chunk metadata, and embedding availability all being present and aligned.
- Model access records influence what users can see or manage in the broader application experience.